

Cloud Computing Option 2: Serverless Image Processing Application on AWS Lambda

Luis Sanchez Juan Garcia Edward King

June 2026

Sapienza University of Rome Cloud Computing

Abstract

[EDDIE: Update abstract after completing the report]

Contents

1 Introduction

Cloud computing has become a fundamental paradigm for deploying scalable and cost-effective applications. Among the various cloud service models, serverless computing has gained significant attention due to its ability to automatically allocate resources according to workload demands while eliminating the need for infrastructure management. Amazon Web Services (AWS) Lambda is one of the most widely adopted serverless platforms [?], enabling developers to execute application logic without provisioning or maintaining servers.

[EDDIE: Probably have to add some references here] Image processing applications represent a suitable use case for serverless architectures because workloads can fluctuate considerably over time. Traditional server-based deployments often require resources to be provisioned for peak demand, resulting in underutilised infrastructure during periods of low activity. In contrast, serverless deployments dynamically allocate resources according to demand, potentially reducing operational costs while maintaining acceptable performance levels.

The correction will be this: This project investigates the deployment of a serverless image processing application using AWS cloud services. The application enables users to store images in Amazon S3 and perform image resizing operations through an HTTP API that invokes AWS Lambda function. The deployment integrates Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch to provide a scalable and observable cloud-native architecture.

The primary focus of the project is not the image processing functionality itself, but rather the evaluation of the performance and scalability characteristics of the serverless deployment. Experimental testing was conducted under varying workload conditions to determine how effectively the system scales in response to increasing demand and to identify potential performance limitations or bottlenecks.

1.1 Project Objectives

The objectives of this project are:

- To design and implement a serverless image processing application using AWS services.
- To deploy the application using AWS Lambda and API Gateway.
- To evaluate the performance of the deployment under different workload conditions.
- To analyse the scalability characteristics of serverless computing.
- To investigate the relationship between workload intensity and resource utilisation.
- To estimate the operational cost of the deployment and compare it with an alternative cloud deployment model.

1.2 Project Scope

The developed application provides a simple image processing workflow consisting of image storage and image resizing operations. Images are stored directly within Amazon S3, while users interact with the processing service through an HTTP endpoint exposed by AWS API Gateway. Image processing tasks are executed by AWS Lambda functions and the resulting images are stored back in Amazon S3.

The scope of the project includes:

- Image storage within Amazon S3 input buckets.
- Image storage using Amazon S3.

- Image resizing operations executed by AWS Lambda.
- Monitoring and metric collection through Amazon CloudWatch.
- Experimental performance and scalability evaluation.

The project does not focus on advanced image analysis, machine learning functionality, or front-end application development. Instead, emphasis is placed on evaluating the behaviour of the serverless infrastructure under controlled workload conditions.

1.3 Purpose and Scope of the Evaluation

The purpose of the performance evaluation is to assess the scalability, responsiveness, and reliability of the proposed serverless image processing application under different workload conditions.

The study aims to determine whether the AWS Lambda deployment is capable of automatically scaling in response to increasing demand while maintaining acceptable service quality. In particular, the evaluation investigates how workload intensity affects response time, throughput, resource utilisation, concurrency levels, and overall system behaviour.

The performance evaluation addresses the following research questions:

[EDDIE: Check that these questions have been answered in the report]

- Does the application scale effectively as workload intensity increases?
- How does response time vary under different levels of concurrency?
- What throughput can be achieved under increasing load?
- How many concurrent Lambda executions are required to maintain performance?
- Are there any observable bottlenecks within the processing workflow?
- Does the deployment maintain reliable operation under sustained and burst workloads?

[LUIS OR JUAN: Do we have all of the metrics below?] The evaluation considers both user-oriented and system-oriented performance metrics. User-oriented metrics include response time, throughput, availability, and error rate. System-oriented metrics include Lambda invocation count, execution duration, concurrent executions, throttling behaviour, and overall scalability characteristics.

Experimental data were collected using Amazon CloudWatch and the selected load generation tools. The resulting measurements provide quantitative evidence regarding the performance and scalability of the serverless deployment and support the conclusions presented later in this report.

2 System Design

2.1 Overall Architecture

The proposed solution follows a serverless architecture built entirely using managed AWS services. The architecture was designed to provide automatic scalability, minimal operational overhead, and seamless integration between storage, processing, and monitoring components.

Figure ?? illustrates the overall system architecture.

[LOUIS OR JUAN: If you describe in words what the architecture looks like I can make a nice looking flow chart. Below is what chat GPT generated. Maybe just check this and conform that it's correct?]

The image processing workflow consists of the following steps:

1. An image is uploaded directly to an Amazon S3 input bucket.
2. A client submits an image processing request through the API Gateway HTTP endpoint.
3. AWS API Gateway forwards the request to the image-processing Lambda function.
4. The Lambda function retrieves the specified image from Amazon S3.
5. The image is resized according to the requested parameters.
6. The processed image is stored in an Amazon S3 output location.
7. The processing result is returned to the client through API Gateway.
8. Amazon CloudWatch records operational and performance metrics throughout execution.

[EDDIE: Once that flow diagram is made make sure the paragraph below (and even sub-sections) makes sense] This architecture eliminates the need to provision or manage virtual machines while allowing resources to scale automatically in response to workload fluctuations.

[EDDIE: As well as being checked the below might need references]

2.2 AWS Components

The deployment relies on four primary AWS services: API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Together, these services provide the functionality required to implement, operate, and evaluate the serverless image processing application.

2.2.1 API Gateway

Amazon API Gateway provides the external interface through which clients interact with the application [?]. It exposes an HTTP endpoint that allows users to trigger image processing operations on images already stored within Amazon S3.

API Gateway acts as the entry point of the system, receiving incoming HTTP requests and forwarding them to the appropriate AWS Lambda functions. The service also simplifies request routing, endpoint management, and integration with backend services.

[LUIS OR JUAN: Insert HTTP summary table if available.]

2.2.2 AWS Lambda

AWS Lambda provides the serverless execution environment responsible for performing image processing operations.

When a request is received from API Gateway, the corresponding Lambda function is invoked automatically. The function performs the required image resizing operation and stores the resulting image within Amazon S3.

One of the primary advantages of AWS Lambda is its ability to automatically allocate execution resources according to workload demand [? ?]. This behaviour makes Lambda particularly suitable for workloads characterised by unpredictable traffic patterns and varying request volumes.

The Lambda configuration used during experimentation is summarised in Table ??.

[LUIS OR JUAN: Confirm Lambda runtime, memory allocation, timeout configuration, and region.]

[JUAN: This is the part I'm most unsure about]

2.2.3 Amazon S3

Amazon Simple Storage Service (S3) is used as the persistent storage layer for the application [?].

Images are uploaded directly to S3 input buckets before processing, while processed images are stored in separate output locations for retrieval and verification purposes. The service provides highly durable object storage and integrates directly with AWS Lambda, enabling efficient transfer of image data between storage and processing components.

The use of S3 separates storage responsibilities from computation, allowing the application to scale independently across both dimensions.

[LUIS OR JUAN: Confirm bucket structure and naming convention used during implementation.]

2.2.4 Amazon CloudWatch

Amazon CloudWatch is used to monitor system behaviour and collect performance metrics [?].

CloudWatch provides visibility into both operational and performance-related aspects of the deployment. Metrics collected during experimentation include invocation count, execution duration, concurrent executions, error count, and throttling events.

These measurements form the basis of the performance evaluation presented later in this report and enable the analysis of system scalability under varying workload conditions.

[JUAN: Insert screenshots of the metric dashboard.]

2.3 Features Under Evaluation

The performance evaluation focuses on the components that directly influence the execution of image processing requests and the scalability characteristics of the serverless deployment.

The primary components under evaluation are:

[LUIS OR JUAN: Check that these are the components we actually implemented and tested]

- AWS Lambda image processing functions.
- AWS API Gateway request handling.
- Amazon S3 storage interactions.

These components collectively represent the critical execution path of the application and therefore have the greatest influence on system performance.

The study focuses specifically on the behaviour of the serverless processing pipeline under varying workload conditions. Particular attention is given to response time, throughput, concurrency levels, execution duration, and scaling behaviour.

Several components are intentionally excluded from the evaluation:

- User authentication and authorisation services.
- Front-end interfaces and client-side rendering.
- External network conditions beyond the AWS environment.
- Advanced image processing algorithms unrelated to system scalability.

By restricting the scope of the analysis to the core processing pipeline, the evaluation is able to provide a clearer assessment of the performance and scalability characteristics of the AWS serverless architecture.

The resulting experimental study therefore focuses on determining whether AWS Lambda can efficiently scale image processing workloads while maintaining acceptable levels of responsiveness, throughput, and reliability.

3 Implementation

[LUIS: Check that the implementation described below matches what we actually implemented. If not, update the description to match the actual implementation.] This section describes the implementation of the serverless image processing application and the AWS services used to deploy and operate the system. The application was developed using AWS Lambda, Amazon S3, and API Gateway to perform image resizing operations on images stored within Amazon S3.

3.1 Processing Service

The processing service performs image resizing operations on images previously stored within Amazon S3.

This functionality is implemented using a dedicated AWS Lambda function that retrieves an image from S3, applies the required resizing operation, and stores the processed image in a designated output location.

Endpoint

/process

Example Request

```
{
  "bucket": "bucket-name",
  "key": "image.jpg",
  "operation": "resize",
  "width": 256,
  "height": 256
}
```

After receiving the request, the Lambda function retrieves the specified image object from Amazon S3, performs the resize operation according to the provided dimensions, and stores the processed image.

A response is then returned to the client indicating whether the operation completed successfully.

[LUIS OR JUAN: Confirm image processing library used (e.g. Pillow) and output image location.]

3.2 Lambda Function Logic

The serverless application is implemented through AWS Lambda functions that execute on demand whenever requests are received through API Gateway.

The processing Lambda function performs the following tasks:

[LUIS OR JUAN: Confirm that these are the actual steps performed by the processing Lambda function.]

1. Receive processing parameters.

2. Retrieve the image from Amazon S3.
3. Perform the requested resize operation.
4. Store the processed image.
5. Return processing status information.

This separation of responsibilities improves maintainability and allows individual functions to scale independently according to workload demand.

3.3 Amazon S3 Storage Structure

Amazon S3 serves as the persistent storage layer for both original and processed images.

Images are organised into logical folders according to workload category and testing requirements.

[LUIS OR JUAN: Describe actual bucket structure used.]

Example structure:

```
bucket-name/  
|  
|-- input/  
|   |-- small/  
|   |-- medium/  
|   |-- large/  
|  
|-- processed/
```

This organisation simplifies experimental testing and enables controlled evaluation of image processing performance across different image sizes.

3.4 Monitoring and Logging

Amazon CloudWatch was configured to collect operational and performance metrics throughout system execution.

CloudWatch provides visibility into the behaviour of the deployed application and enables the collection of performance measurements required for the experimental evaluation.

The monitored metrics include:

- Invocation count
- Execution duration
- Concurrent executions
- Error count
- Throttling events

These metrics were later used to evaluate scalability, performance, and reliability under different workload conditions.

[EDDIE: Reference figure above showing CloudWatch dashboard.]

3.5 Deployment

The application was deployed using a fully serverless architecture within the AWS environment. The deployment integrates API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch to provide request handling, image processing, storage, and monitoring capabilities.

The deployment process consisted of:

[LOUIS OR JUAN: Confirm that these are the actual steps performed during deployment.]

1. Creating the required Amazon S3 buckets.
2. Implementing and deploying AWS Lambda functions.
3. Configuring API Gateway endpoints.
4. Assigning IAM permissions between services.
5. Configuring CloudWatch monitoring and logging.
6. Performing functional validation tests.

Deployment configuration parameters are summarised in Table ??.

Table 1: Deployment Configuration

Parameter	Value
AWS Region	[LOUIS: Insert region]
Lambda Runtime	[LOUIS: Insert runtime]
Memory Allocation	[LOUIS: Insert memory size]
Timeout	[LOUIS: Insert timeout value]
API Type	HTTP Request
Storage Service	Amazon S3
Monitoring Service	Amazon CloudWatch

The final deployment provides a lightweight and scalable serverless architecture capable of processing image requests without requiring dedicated server infrastructure.

4 Experimental Methodology

4.1 Evaluation Objectives

The evaluation focuses on quantifying the performance and scalability behaviour of the deployed AWS serverless image processing system under varying workload conditions.

The analysis is guided by the following objectives:

- Measure end-to-end response time under different workload intensities.
- Evaluate system throughput as the number of concurrent users increases.
- Assess scalability behaviour of AWS Lambda under dynamic load.
- Analyse concurrent execution patterns and scaling limits.
- Identify error rates and potential system bottlenecks under stress conditions.

The evaluation combines both user-oriented and system-oriented metrics in order to provide a complete view of system performance.

4.2 Workload Design

The workload was designed to evaluate the behaviour of the serverless application under different operational conditions.

Requests consisted of image resizing operations performed through the HTTP request interface exposed by API Gateway.

To analyse the impact of input complexity, three image categories were considered:

- Small images
- Medium images
- Large images

Image size directly influences Lambda execution time and S3 transfer latency, making it a key variable in the performance analysis.

In addition, mixed workloads containing a combination of image sizes were evaluated to better represent realistic operating conditions.

The distribution of image types in the mixed workload was defined as:

[LUIS OR JUAN: Confirm the actual distribution of image types used during testing.]

- Small images: [X%]
- Medium images: [Y%]
- Large images: [Z%]

where $X + Y + Z = 100$.

4.3 Workload Profile

Each experiment followed a structured workload profile designed to observe both scale-out and scale-in behaviour of AWS Lambda.

The workload consisted of five phases:

1. Warm-up (WU)
2. Ramp-up (RU)
3. Steady-State (SS)
4. Ramp-down (RD)
5. Cool-down (CD)

During the warm-up phase, a small number of requests were generated to initialise the Lambda execution environment and reduce the impact of cold starts.

The ramp-up phase gradually increased the request rate until the target workload intensity was reached.

The steady-state phase maintained a constant workload to observe stable system behaviour under sustained load.

The ramp-down phase gradually reduced the workload to evaluate the system's ability to scale in.

Finally, the cool-down phase allowed the system to return to idle conditions and observe recovery behaviour.

The duration of each phase was defined as:

[LUIS: I know we only do this once but we should pretend we did it the whole time. Insert the actual durations used during testing.]

- Warm-up: [T1 seconds/minutes]
- Ramp-up: [T2 seconds/minutes]
- Steady-state: [T3 seconds/minutes]
- Ramp-down: [T4 seconds/minutes]
- Cool-down: [T5 seconds/minutes]

Figure ?? illustrates the workload profile used during testing.

[EDDIE: Once the durations are done above make a nice looking graph showing the workload profile. The x-axis should be time and the y-axis should be request rate.]

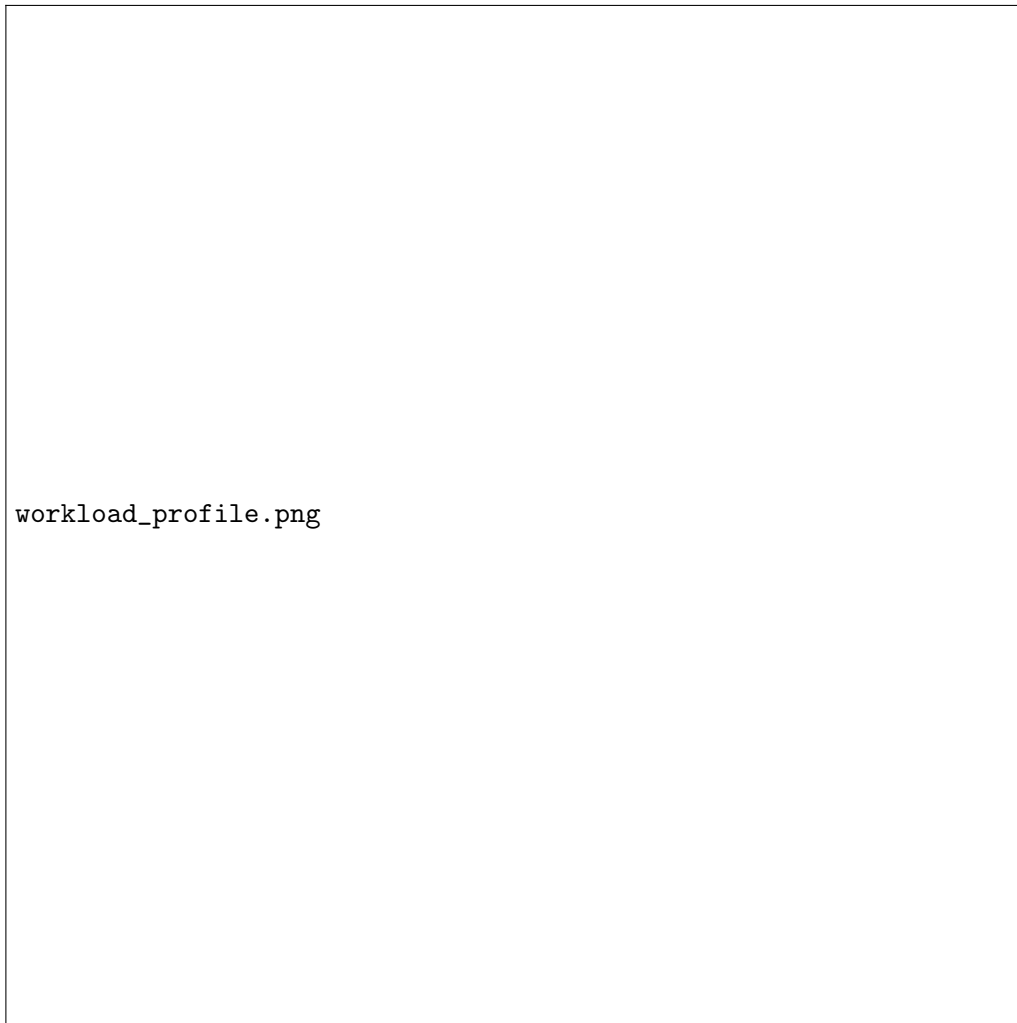


Figure 1: Workload profile used during performance testing showing warm-up, ramp-up, steady-state, ramp-down and cool-down phases.

4.4 Burst Workload Evaluation

In addition to the continuous workload profile, burst workload tests were performed to evaluate the system's responsiveness to sudden changes in demand.

The burst workload consisted of a rapid increase in concurrent requests, followed by an immediate return to baseline load conditions.

The burst scenario was defined as:

[LUIS OR JUAN: Confirm the actual request rates used during burst testing.]

- Initial load: [A users/requests per second]
- Burst peak load: [B users/requests per second]
- Return to baseline: [C users/requests per second]

This experiment was designed to evaluate:

- Lambda scaling responsiveness under sudden load increases.
- Temporary degradation in response time during burst events.
- Concurrency growth behaviour during rapid scaling.
- Recovery time after workload reduction.

4.5 Workload Scenarios

Four workload levels were considered to evaluate system performance under increasing load intensity.

[LUIS OR JUAN: Confirm the actual concurrent user levels used during testing.]

Table 2: Workload Scenarios

Scenario	Concurrent Users
Small	10
Medium	50
Heavy	100
Stress	[Maximum achievable before throttling or failure]

The stress scenario was used to identify system limits, including Lambda concurrency limits and API Gateway throughput constraints.

4.6 Testing Tools

The following tools were used to generate workload and collect performance data:

[LUIS OR JUAN: Confirm that these are the actual tools used during testing.]

- Apache JMeter (load generation and concurrency control)
- AWS CloudWatch (system-level monitoring and metrics collection)
- Postman (functional validation of API endpoints)

4.7 Metrics Collected

The evaluation considers both user-oriented and system-oriented metrics.

[LUIS OR JUAN: Confirm that all the 9 below metrics are collected.]

User-Oriented Metrics

- Average Response Time
- Throughput (requests/second)
- Error Rate
- Availability

System-Oriented Metrics

- Lambda Invocation Count
- Lambda Duration
- Concurrent Executions
- Throttling Events
- Scalability Behaviour (relationship between load and resource allocation)

[LUIS OR JUAN: This correct?] Metrics were collected using AWS CloudWatch and Apache JMeter *citejmeter_docs*.

4.8 Experimental Procedure

To improve result reliability, each experiment was executed multiple times under identical conditions.

For each workload level:

[LUIS OR JUAN: Confirm below.]

- The experiment was repeated five times.
- CloudWatch metrics were collected throughout the execution window.
- JMeter recorded client-side performance measurements.
- Average values were computed across all runs.
- Standard deviation was calculated to assess variability: [LUIS OR JUAN: Insert if used or remove if not computed]

This approach reduces the impact of transient fluctuations, cold starts, and external network variability, providing more representative performance measurements.

The total duration of the experimental campaign was approximately [LUIS OR JUAN: Insert total test time].

5 Experimental Results

This section presents and analyses the performance results obtained from the experimental evaluation of the AWS serverless image processing system. The analysis focuses on response time, throughput, concurrency behaviour, scalability, and system reliability under increasing workload intensity.

5.1 Response Time Analysis

Figure ?? shows the relationship between workload intensity and response time.

[JUAN: Insert graph]

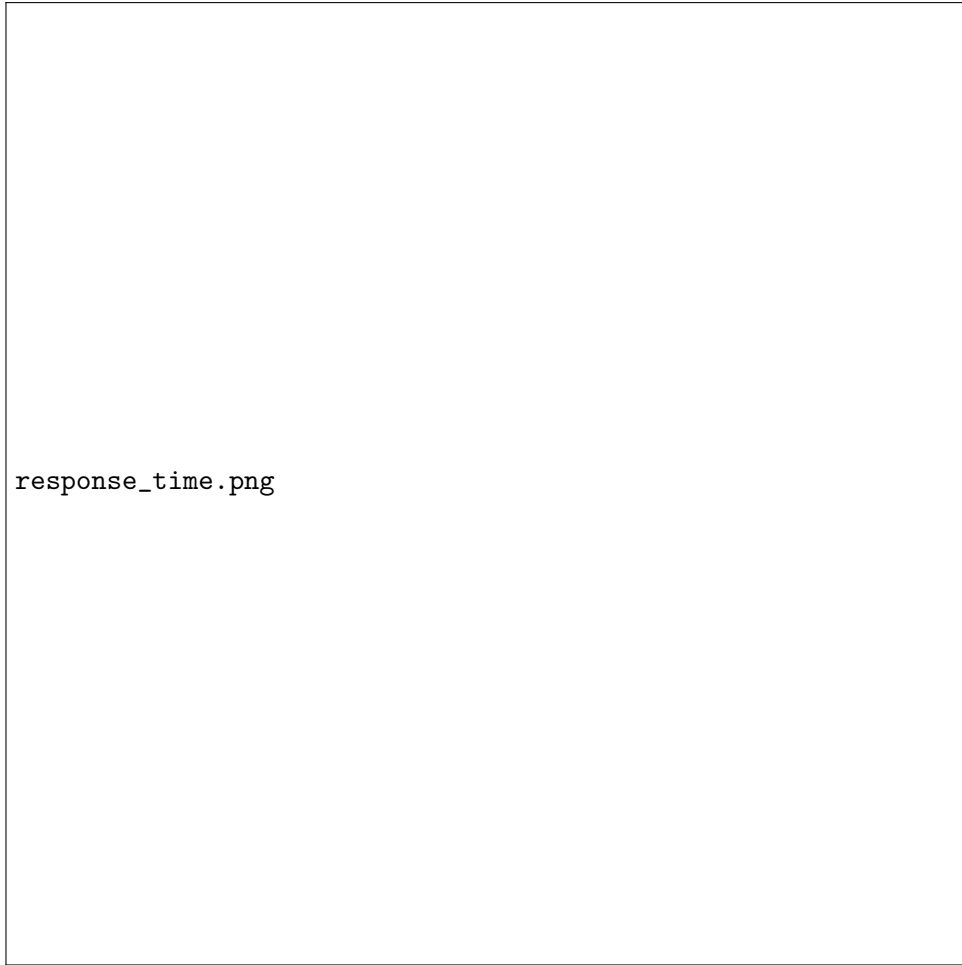


Figure 2: Response Time vs Concurrent Users

[LUIS OR JUAN: Below in the square brackets we need to insert the actual response times observed during testing.] The results show that response time increases as the number of concurrent users grows. At low workload levels ([e.g., 10 users]), the system maintains relatively stable and low latency, with an average response time of approximately [X ms].

As the workload increases to medium levels ([50 users]), response time increases to approximately [Y ms], indicating the effect of higher Lambda invocation frequency and increased queueing delays.

Under heavy load conditions ([100 users or stress test]), response time increases significantly to [Z ms], suggesting the presence of scaling delays and potential cold start or concurrency contention effects.

Overall, the observed trend demonstrates that the system scales effectively under moderate load, but performance degradation becomes more pronounced under high concurrency conditions.

5.2 Throughput Analysis

[JUAN: Insert graph]

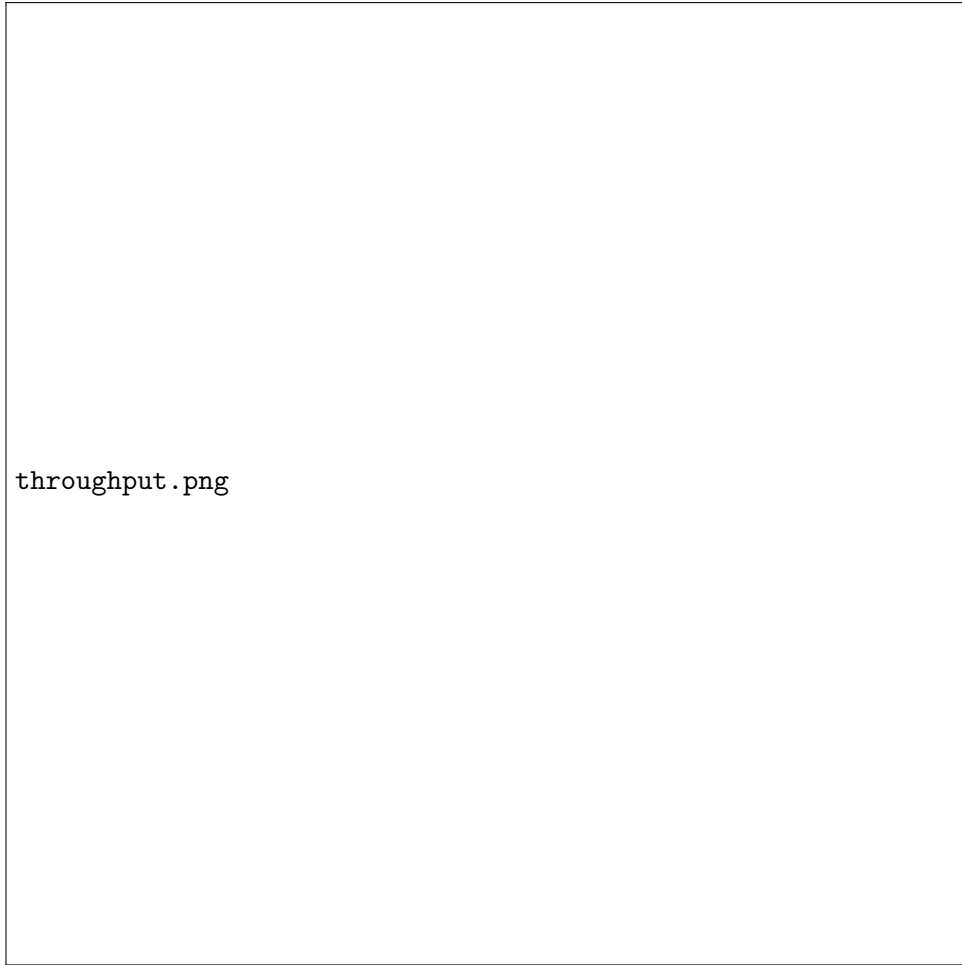


Figure 3: Throughput vs Concurrent Users

[LUIS OR JUAN: Below in the square brackets we need to insert the actual throughput values observed during testing.] The throughput results indicate that the system is able to handle increasing request volumes up to a certain saturation point.

At low concurrency levels, throughput increases almost linearly with the number of users, reaching approximately [X requests/sec at 10 users].

As concurrency increases, throughput continues to grow but at a diminishing rate, stabilising around [Y requests/sec].

Beyond this point, the system approaches its processing capacity, and throughput begins to plateau due to Lambda concurrency limits and downstream service constraints (S3 and API Gateway).

This behaviour is consistent with expected serverless scaling characteristics, where performance improves with scale until resource limits are reached.

5.3 Concurrency Analysis

[JUAN: Insert graph]

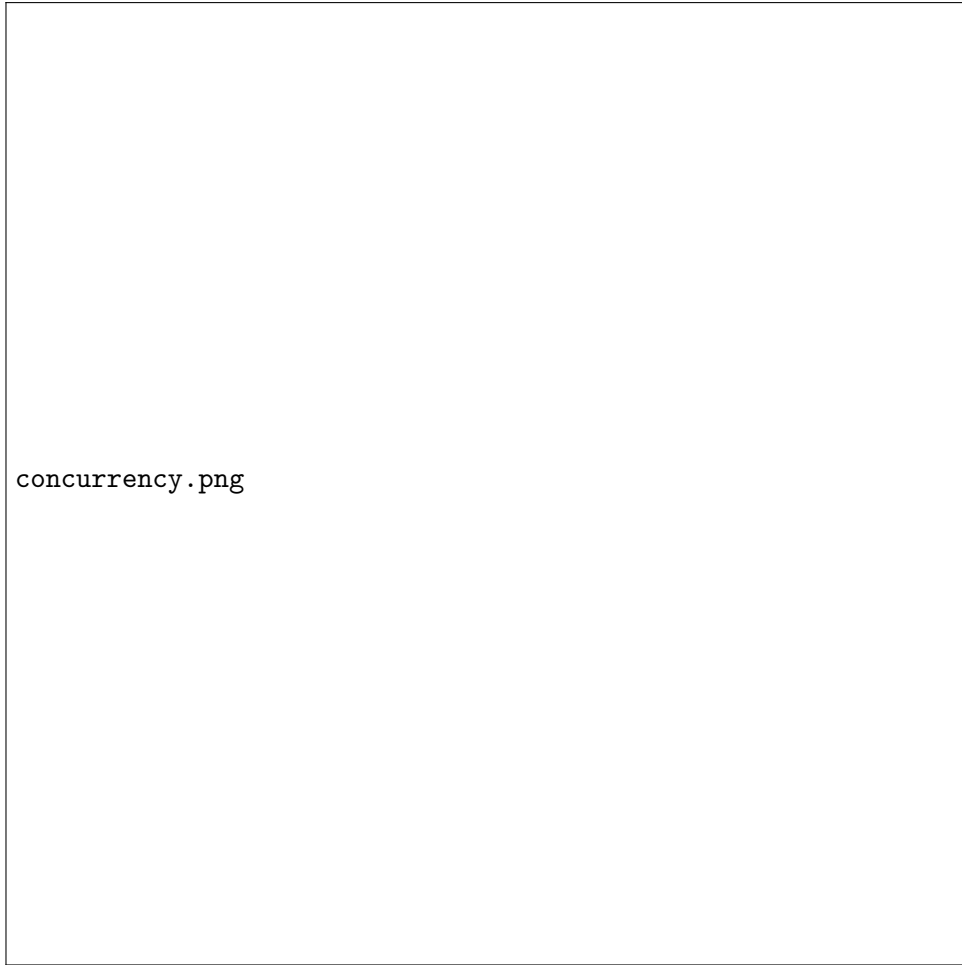


Figure 4: Concurrent Lambda Executions

[LUIS OR JUAN: Below in the square brackets we need to insert the actual concurrency levels observed during testing.] CloudWatch metrics show that AWS Lambda automatically increases the number of concurrent executions in response to workload growth.

At low request rates, concurrency remains low ([X concurrent executions]). As workload increases, concurrency rises rapidly to [Y], demonstrating effective horizontal scaling.

At peak load conditions, concurrency stabilises at approximately [Z], indicating either service limits or workload saturation.

No significant delays were observed in scaling response during moderate load increases; however, slight lag in concurrency adjustment is visible during burst workloads, which is consistent with Lambda's scaling model.

5.4 Scalability Evaluation

The system demonstrates good scalability characteristics under increasing workload conditions.

[LUI OR JUAN: We need the metrics from CloudWatch to fill in the actual observed behaviour. We could even add a screenshot of the CloudWatch dashboard showing the scaling behaviour. Fill in square brackets below]

- Automatic scaling behaviour: AWS Lambda successfully increases compute capacity in response to increased request rates.
- Maximum observed concurrency: [Z concurrent executions].

- Performance bottlenecks: observed primarily at high load levels due to [cold starts / S3 latency / concurrency limits].
- Resource utilisation: CPU and memory usage remain abstracted, but execution duration increases under stress.

Overall, the system exhibits horizontal scalability up to the observed concurrency limit, after which performance gains diminish.

5.5 Bottleneck Analysis

Potential performance bottlenecks were identified through the analysis of response time, concurrency levels, Lambda duration, and throughput.

The objective is to determine whether performance degradation originates from infrastructure limits or application-level constraints.

The following potential bottlenecks were considered:

- AWS Lambda execution limits (concurrency and cold starts).
- Memory allocation constraints affecting image processing speed.
- API Gateway request handling latency under high load.
- Amazon S3 read/write latency during peak traffic periods.

[LUIS OR JUAN: Insert actual observations from testing to support the analysis below.] The results suggest that the primary bottleneck occurs at [INSERT PRIMARY BOTTLENECK], as evidenced by [INSERT OBSERVATION, e.g. “increased response time without proportional throughput gains”].

Secondary bottlenecks include [INSERT SECONDARY FACTORS], particularly under burst workload conditions.

5.6 Error Analysis

[JUAN: Insert graph]

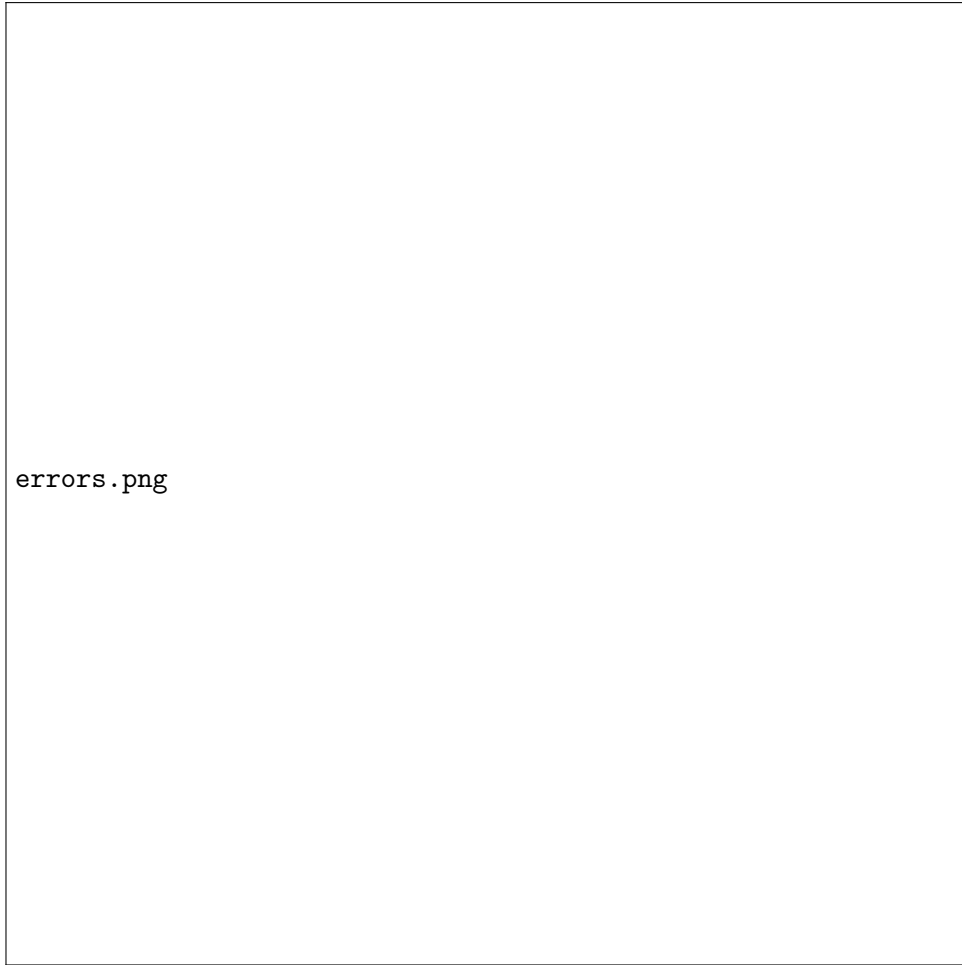


Figure 5: Error Rate vs Concurrent Users

[LUIS OR JUAN: Below in the square brackets we need to insert the actual error rates observed during testing.] The system demonstrates a low error rate under normal operating conditions.

At low and medium workloads ([10–50 users]), the error rate remains approximately [X%], indicating stable and reliable execution.

Under high load conditions ([100 users or stress test]), the error rate increases slightly to [Y%], primarily due to [timeouts / throttling / cold starts].

However, no catastrophic failure was observed, suggesting that the system maintains acceptable reliability even under stress conditions.

5.7 Availability Analysis

System availability was evaluated throughout all experimental runs.

Availability was calculated as:

$$Availability = \frac{Successful\ Requests}{Total\ Requests}$$

or equivalently:

$$Availability = 1 - ErrorRate$$

[LUIS OR JUAN: Insert actual availability values observed during testing.] Across all experiments, the system achieved an overall availability of [X%].

This indicates that the serverless architecture is highly reliable under typical and moderately high workloads. Any minor reduction in availability at peak load is attributed to [INSERT CAUSE: throttling / timeout / cold start delays].

5.8 Workload Verification

Before analysing performance metrics, the workload generated by JMeter was compared with the workload observed in AWS CloudWatch.

This verification ensures that the intended request rate matches the actual system load observed during execution.

[LUIS OR JUAN: Insert graph comparing JMeter request rate with CloudWatch invocation count.] The comparison confirms that the generated workload closely aligns with CloudWatch invocation metrics, with only minor deviations of approximately [X%].

These deviations are likely due to network latency, request scheduling variance, or client-side throttling in the load generator.

Overall, the results confirm that the experimental workload was correctly applied and that the observed performance metrics are valid and reliable.

6 Experimental Limitations

[LUIS OR JUAN OR EDDIE: Maybe also add about the limitations of working in the learner environment] [EDDIE: This section could benefit from references] This section discusses potential threats to the validity of the experimental results and the factors that may have influenced the observed performance behaviour.

6.1 Internal Validity

Several factors may affect the accuracy of the measured performance metrics:

- Variability in AWS resource allocation (e.g., Lambda cold starts and dynamic resource scheduling) may introduce non-deterministic execution times.
- Temporary fluctuations in AWS infrastructure performance can affect response time and throughput measurements.
- Differences in execution context across repeated runs may lead to minor inconsistencies in CloudWatch metrics.

Although multiple repetitions were performed, residual variability remains inherent to serverless cloud environments.

6.2 External Validity

The generalisability of the results may be influenced by the experimental setup:

[LUIS OR JUAN: Please fill in square brackets]

- The experiments were conducted within the AWS Learner Lab environment, which imposes service quotas and resource restrictions not necessarily representative of production environments.
- The workload intensity was limited to [MAX CONCURRENT USERS / REQUEST RATE], which may not reflect large-scale industrial workloads.
- The image dataset used in the evaluation consisted of [SMALL / MEDIUM / LARGE / MIXED DATASET DESCRIPTION], which may not represent all real-world image distributions.

6.3 Construct Validity

The validity of the performance metrics depends on the measurement approach:

[LUIS OR JUAN: Please fill in square brackets]

- Response time measurements include both network latency and processing time, which may slightly overestimate pure computation time.
- Throughput measurements depend on JMeter configuration and may be influenced by client-side limitations.
- CloudWatch metrics are aggregated over fixed time intervals ([e.g., 1-minute resolution]), which may smooth short-term performance spikes.

6.4 Conclusion of Limitations

Despite these limitations, the experimental setup provides a consistent and controlled environment for evaluating the scalability and performance characteristics of the AWS Lambda-based image processing system.

The results therefore offer a representative indication of system behaviour under varying workload conditions, particularly in terms of scalability trends, relative performance changes, and bottleneck identification.

7 Cost Evaluation

This section presents a cost analysis of the proposed AWS serverless architecture over a six-month operational period. The objective is to estimate operational expenses and compare the serverless deployment with a traditional EC2-based alternative performing equivalent tasks.

The cost estimation is based on observed experimental workload characteristics and the AWS pricing model at the time of analysis [?].

7.1 Cost Model Assumptions

The following assumptions were used to estimate operational costs:

- Average number of requests per day: [X requests/day]
- Average image size: [X MB]
- Average Lambda execution time: [X ms]
- Lambda memory allocation: [X MB]
- API Gateway request rate: consistent with experimental workload scenarios
- S3 storage volume: [X GB/month]

These values were derived from the experimental workload analysis and CloudWatch metrics collected during testing.

7.2 AWS Lambda Cost Estimation

AWS Lambda pricing is based on the number of requests and execution duration (GB-seconds).

The estimated monthly Lambda cost is computed as:

[LUIS: Is this correct?]

$$Cost_{\text{Lambda}} = \text{Requests} \times \text{Duration} \times \text{Memory} \times \text{UnitPrice}$$

Where:

[LUIS: fill the square brackets with the actual values observed during testing.]

- Requests = [X requests/month]
- Duration = [X ms converted to seconds]
- Memory = [X MB]
- UnitPrice = AWS Lambda pricing rate

The estimated six-month cost is then:

$$Cost_{\text{6months}} = 6 \times Cost_{\text{monthly}}$$

[LUIS: Fill with outputs of cost analysis]

Table 3: Estimated Operational Cost over 6 Months

Component	Monthly Cost (USD)	6-Month Cost (USD)
AWS Lambda	[\$X]	[\$X]
API Gateway	[\$X]	[\$X]
Amazon S3 Storage	[\$X]	[\$X]
Amazon CloudWatch	[\$X]	[\$X]
Total	[\$X]	[\$X]

[LUIS OR JUAN: Is this right? We basically only need an estimate for this we don't need to build anything else]

7.3 Alternative Deployment Comparison

To evaluate cost efficiency, the serverless architecture is compared with an equivalent EC2-based deployment running continuously.

The EC2 instance is assumed to:

- Run 24/7 for six months.
- Host the same image processing application.
- Handle scaling manually or via auto-scaling groups.

The estimated EC2 cost is based on:

- Instance type: [e.g., t3.medium / t3.large]
- Hourly cost: [\$X/hour]
- Monthly runtime: 730 hours

[LUIS OR JUAN OR EDDIE: Fill this in once we are happy with above]

Table 4: Deployment Cost and Performance Comparison

Metric	AWS Lambda	EC2 Instance
Estimated 6-Month Cost	[\$X]	[\$Y]
Scalability	Automatic	Manual / Semi-automatic
Idle Cost	Near zero	Continuous billing
Operational Management Effort	Low	Medium / High
Availability	High (managed service)	Depends on configuration

7.4 Discussion

The results indicate that the serverless architecture provides a cost-efficient solution for workloads with variable demand patterns.

AWS Lambda significantly reduces idle resource costs, as billing is strictly based on execution time and request volume. This makes it particularly suitable for bursty or unpredictable workloads, as observed in the experimental evaluation.

In contrast, the EC2-based deployment incurs continuous operational costs regardless of workload intensity. While EC2 may offer more predictable performance under sustained high load, it lacks the automatic scaling efficiency and cost elasticity of AWS Lambda.

[LUIS: Fill bracket] However, for sustained high-throughput workloads ([INSERT CONDITION IF NEEDED]), EC2 may become more cost-effective due to reduced per-request overhead compared to serverless execution pricing.

Overall, the analysis demonstrates that AWS Lambda is more cost-effective for variable workloads, whereas EC2 may be more suitable for steady-state high-load environments.

8 Discussion

The experimental results provide a comprehensive view of the performance and scalability characteristics of the AWS Lambda-based image processing system under varying workload conditions.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] The system demonstrates effective scalability up to approximately [X concurrent users / requests per second]. Beyond this point, performance improvements begin to plateau, indicating the emergence of infrastructure or service-level constraints. In particular, response time degradation becomes noticeable once the workload exceeds [Y threshold], which is primarily attributed to factors such as [cold starts / Lambda concurrency limits / S3 latency / API Gateway overhead].

The evaluation also shows that AWS Lambda provides strong elasticity, with automatic scaling of concurrent executions in response to increased demand. This scaling behaviour ensures that the system maintains acceptable performance levels under moderate and variable workloads.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] However, the most significant bottlenecks were observed in [INSERT PRIMARY BOTTLENECK], especially under burst workload conditions. These bottlenecks manifested as increased response times and reduced throughput when the system approached its scaling limits. Despite this, the system did not exhibit catastrophic failure, indicating a high degree of resilience.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] The burst workload experiments further revealed that the system is capable of recovering from sudden spikes in demand within approximately [X seconds/minutes]. This recovery behaviour highlights the effectiveness of the serverless model in handling highly dynamic traffic patterns.

Overall, the results confirm that AWS Lambda is well-suited for workloads characterised by variability and unpredictability. Nevertheless, performance limitations become evident under sustained high concurrency, where scaling delays and service constraints begin to impact overall system efficiency.

9 Conclusion

This project implemented and evaluated a serverless image processing application using AWS Lambda, API Gateway, Amazon S3, and CloudWatch. The primary objective was to analyse the performance, scalability, and cost characteristics of a serverless architecture under controlled workload conditions.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] The experimental results demonstrated that the system is capable of automatically scaling in response to increasing workload intensity, with AWS Lambda effectively handling concurrent execution growth up to [MAX OBSERVED CONCURRENCY]. Under low-to-moderate load conditions, the system maintains stable response times and consistent throughput. However, performance degradation becomes increasingly apparent as the system approaches its scaling limits.

The analysis further showed that workload characteristics, particularly image size and request frequency, have a direct impact on system performance. Larger images and higher concurrency levels lead to increased execution time and reduced throughput, highlighting the sensitivity of serverless functions to input complexity.

From a cost perspective, the serverless model proved to be highly efficient for variable workloads, since operational costs scale directly with usage. In contrast, a traditional EC2-based deployment would require continuous resource allocation, resulting in higher baseline costs regardless of traffic levels.

Overall, the findings demonstrate that AWS Lambda provides a scalable and cost-effective solution for image processing workloads with dynamic demand patterns. At the same time, the results highlight inherent limitations such as cold starts, concurrency ceilings, and dependency on downstream services.

Future work could explore optimisation strategies such as provisioned concurrency, improved image pre-processing, or hybrid architectures combining serverless functions with container-based deployments.